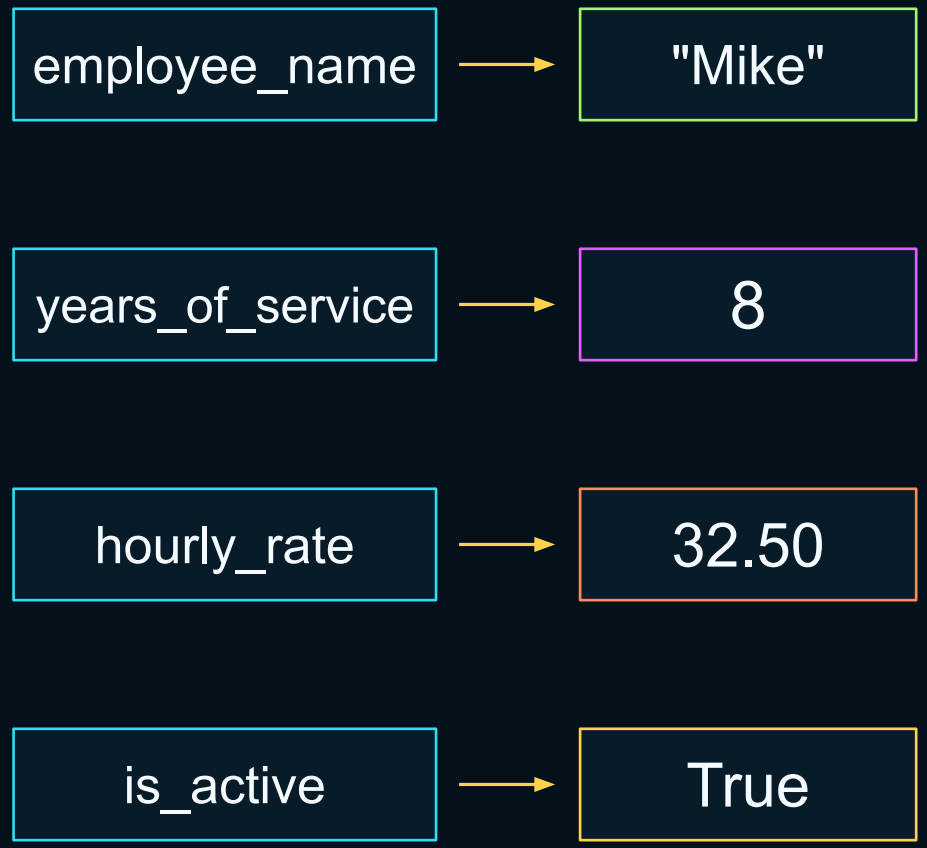
VARIABLES AND VALUES

Giving useful names to data so programs can do work



30 min instruction • 30 min guided variable lab

```
name = value
```

INSTRUCTION

Values: the data Python works with

A value is a piece of information a program can store, display, compare, or calculate with.

"Sensor Upgrade"	6	125000.00	False
text value	whole number	decimal number	true / false

Python values have types. The type tells Python what kind of thing the value is.

The same bits can mean different things; Python keeps track of meaning using types.

INSTRUCTION

Variables: useful names for values

A variable is a name a program uses to refer to a value.

```
project_name = "Sensor Upgrade"
```

Think: the name is a label you can use later. The value is the information being labeled.

Name	Assignment	Value
project_name	=	"Sensor Upgrade"

INSTRUCTION

Assignment: read the equals sign as gets

In Python, assignment attaches a name to a value. It is not the same as an algebra equation.

```
team_size = 6
```

team_size gets 6 or team_size is assigned 6.

INSTRUCTION

Example: meaningful names make data readable

The names tell a human what the values are supposed to represent.

```
employee_name = "Mike"
years_of_service = 8
hourly_rate = 32.50
is_active = True
```

Text: employee_name

Whole number: years_of_service

Decimal number: hourly_rate

True/false: is_active

Good variable names reduce the amount of guessing the reader has to do.

INSTRUCTION

Variable names: legal vs readable

Python has rules; teams also use conventions so code stays clean.

GOOD PYTHON STYLE	AVOID
employee_name	employeename
years_of_service	yrs
hourly_rate	Hourly Rate
is_active	1project
project_budget	class

Beginner rule: use lowercase words separated by underscores, and make the name mean something.

INSTRUCTION

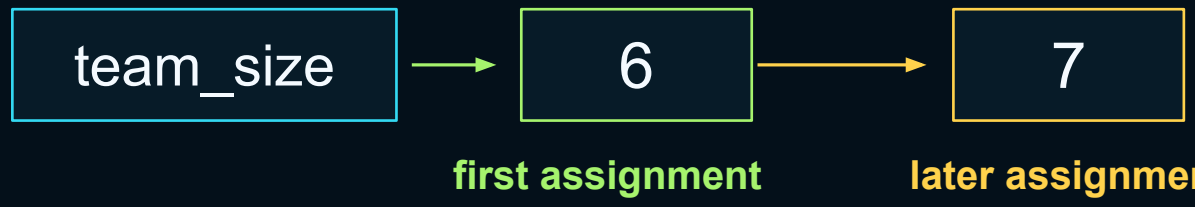
Reassignment: a name can point to a new value

Programs change state over time. The same variable name can be assigned again.

```
team_size = 6
print(team_size)

team_size = 7
print(team_size)
```

```
6
7
```



INSTRUCTION

Four starter types

These cover a lot of beginner Python data.

str string/text "Mike" "Sensor Upgrade"	int integer/whole number 8 6	float decimal number 32.50 125000.00	bool Boolean True False

Type affects what operations make sense: addition, comparison, display, and storage.

INSTRUCTION

type(): asking Python what kind of value it sees

The type() function is a quick inspection tool.

```
project_name = "Sensor
Upgrade"
team_size = 6
budget = 125000.00
is_complete = False

print(type(project_name))
print(type(team_size))
```



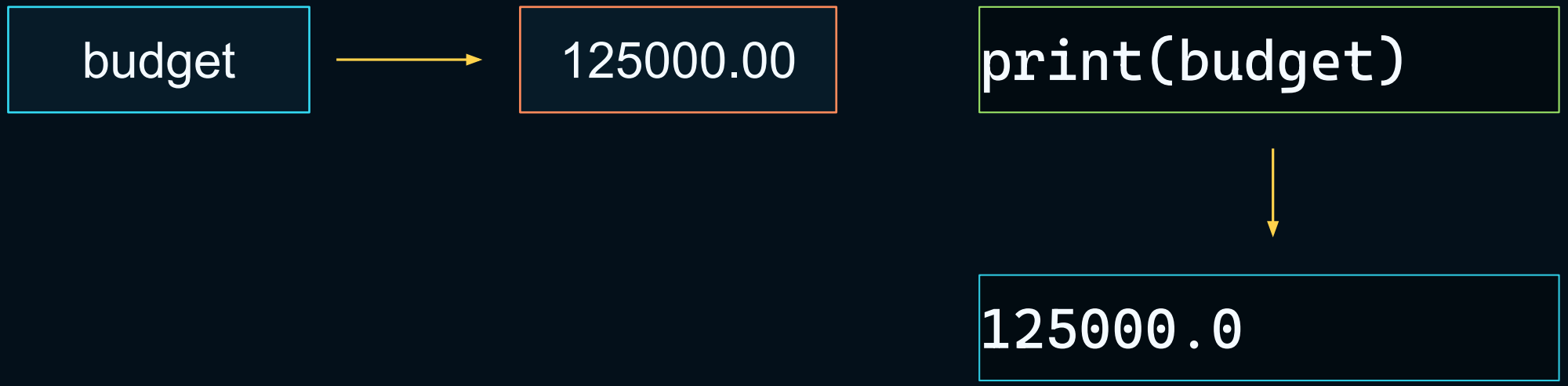
```
<class 'str'>
<class 'int'>
```

type() helps you debug: What did Python think this value was?

INSTRUCTION

Important distinction: name vs value

The variable name is not the data. It is how your code refers to the data.

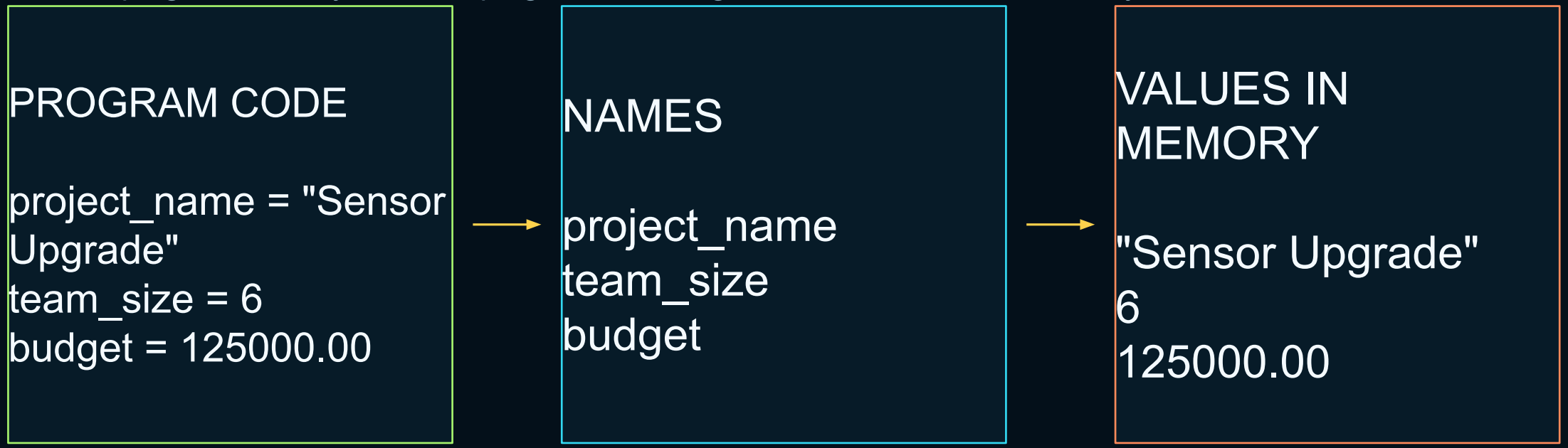


When Python sees budget, it looks up the value currently associated with that name.

INSTRUCTION

Memory mental model: Python manages the details

Values exist in program memory while the program is running. Variables let us reach them by name.

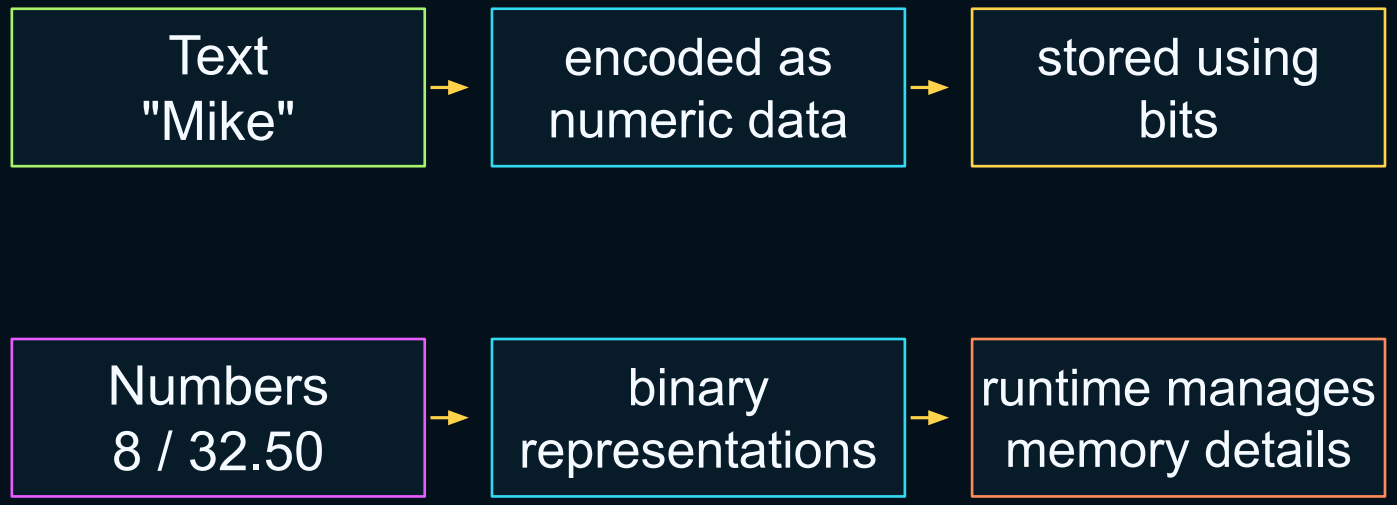


Beginner mental model: names help you find values while your program runs.

INSTRUCTION

Connecting back: values sit on top of bits

Python gives meaning to data that is ultimately represented digitally.



Variables give humans meaningful handles:

employee_name
years_of_service
hourly_rate
is_active

GUIDED EXERCISE

Guided exercise: describe a fictional project

Create four variables, then inspect and update them.

```
project_name = "Sensor Upgrade"
team_size = 6
budget = 125000.00
is_complete = False
```

Task 1: type the variables

Task 2: run the code

Task 3: predict what print() will show

The goal is comfort: create data, name it, print it, inspect it, change it.

GUIDED EXERCISE

Round 1: print each value

Using the variable name prints the value associated with that name.

```
print(project_name)
print(team_size)
print(budget)
print(is_complete)
```



```
Sensor Upgrade
6
125000.0
False
```

Note: Python may display 125000.00 as 125000.0. Same numeric value, shorter display.

GUIDED EXERCISE

Round 2: print each value type

type() shows how Python classifies each value.

```
print(type(project_name))
print(type(team_size))
print(type(budget))
print(type(is_complete))
```



```
<class 'str'>
<class 'int'>
<class 'float'>
<class 'bool'>
```

Ask: which one is text, whole number, decimal number, and true/false?

GUIDED EXERCISE

Round 3: change one value

Reassignment updates what the name points to from that point forward.

```
team_size = 6
print(team_size)

team_size = 8
print(team_size)
```



```
6
8
```

The second assignment does not edit the past. It changes the value used by later lines.

GUIDED EXERCISE

Round 4: create a value from another value

Variables become useful when later code reuses earlier values.

```
budget = 125000.00
team_size = 6

budget_per_person = budget /
team_size

print(budget_per_person)
print(type(budget_per_person))
```



```
20833.333333333332
<class 'float'>
```

Python calculates the expression on the right, then assigns the result to the name on the left.

GUIDED EXERCISE

Round 5: identify what each value represents

Students explain the data using plain language.

```
project_name = "Sensor Upgrade"
team_size = 6
budget = 125000.00
is_complete = False
```

Which value is text?

Which value is a whole number?

Which value is a decimal number?

Which value is true/false?

Plain-language explanation matters. Code is written for people first, machines second.

GUIDED EXERCISE

Common beginner mistakes are normal

The interpreter is strict, but the fixes are usually small.

NAME ERROR

```
print(projct_name)
```

Misspelled variable name.

STRING ERROR

```
project_name = Sensor Upgrade
```

Text needs quotes.

BOOLEAN ERROR

```
is_complete = false
```

Python uses True and False.

Debugging habit: read the line number, check the spelling, check the punctuation.

GUIDED EXERCISE

Mini-lab checklist

Students complete these in the Python environment.

1. **Create** the four fictional project variables
2. **Print** each variable value
3. **Inspect** each value with type()
4. **Change** one value and print it again
5. **Create** a new variable from another variable
6. **Explain** which values are str, int, float, and bool

Instructor goal

Build confidence with named data and immediate feedback.

DISCUSSION

Wrap-up discussion

Students should be able to explain variables in plain language.

What is the difference between a variable name and the stored value?

Why does Python care whether a value is text, a number, or True/False?

What does reassignment do?

How does type() help with debugging?

How do variables connect back to memory and the morning computer model?

Next up: using variables in expressions, decisions, and useful program logic.